



Introduction to Data Science

Assignment 2

Instructors: **Dr. Yaghoobzadeh, Dr. Razavi** TA(s): **Mani Hosseini, Sepanta Ghonoodi, Reihaneh Kharazmi**

Deadline: 2nd Khordad

Introduction

In this project, you are working on **Ashpaz**, a real-time food ordering and delivery platform. You are expected to transition from relational designs toward implementing a functional, live streaming data pipeline.

You are required to utilize **SQL, Kafka, and Spark** as the primary technologies for data ingestion and processing. For auxiliary components—such as the database engine, environment configuration, or visualization libraries, you are permitted to use alternative tools, provided you justify your selection. For instance: *"We utilized Plotly rather than Matplotlib to better accommodate real-time updates for our live dashboard,"* or *"PostgreSQL was selected over MySQL for its robust handling of streaming JSON payloads."* Simply provide a sound technical rationale within your submission. While we have recommended specific tools to ensure manageability, we encourage the use of alternatives that align with your overall architecture.

Upon completion, your pipeline must demonstrate the capability to store restaurant records, process dynamic order and delivery events, identify invalid data streams, and provide actionable real-time business intelligence.



Requirements

PySpark

PySpark is the Python API for Apache Spark, a powerful open-source framework designed for large-scale data processing and analytics. It enables users to leverage Spark's distributed computing capabilities using Python, facilitating tasks such as data transformation, machine learning, and real-time stream processing. We recommend you install and use PySpark in a Linux environment since it's more convenient and user-friendly. Java installation is one of the mandatory things in installing Spark. Try the following command to verify the Java version:



```
java -version
```

In case you do not have Java installed on your system, then install Java before proceeding to the next step. We recommend you use [OpenJDK](#) for a more convenient installation.

Then, you need to install Spark; however, as of v2.2, installing PySpark will install Spark too.

To install PySpark, you can use this [link](#).



Apache Kafka is an open-source distributed event-streaming platform designed for high-throughput, low-latency data pipelines. In this project, Kafka will serve as the ingestion layer for real-time restaurant order events, allowing you to validate, route, and process streaming data efficiently. Kafka uses KRaft mode (Kafka Raft) for metadata management. Note that modern kafka versions (3.3+ and 4.x, which we recommend) no longer require ZooKeeper. If a tutorial or resource you are following still relies on ZooKeeper, it is likely outdated and should not be used for this project. To interact with Kafka using Python, there are two primary client libraries available:

1. **kafka-python**: A pure Python client for Apache Kafka. It's designed to function much like the official Java client, with a Pythonic interface.
2. **confluent-kafka-python**: A Python wrapper around **librdkafka**, a high-performance C library. This client is known for its reliability and performance.

You are allowed to use which of them you like as you see fit, but confluent-kafka is recommended due to its performance and stability. For Kafka installation, configuration, and basic usage, use the video tutorial and PDF guide provided alongside the assignment. These materials are aligned with the version and setup required for this course.

Relational Databases (SQL)

Relational databases are the cornerstone of structured data management, utilizing a predefined schema and SQL (Structured Query Language) for data definition and manipulation. They ensure robust data integrity through ACID compliance, which is essential for the normalization and analytics tasks.

For this project, we recommend utilizing either **PostgreSQL** or **MySQL** as your database engine. PostgreSQL is often preferred for advanced analytical queries and its extensible

nature, while MySQL is widely known for its performance and ease of use in web-based applications.

To interface with these databases using Python, you should install the appropriate connectivity libraries:

PostgreSQL: Use [psycopg2](#), the most popular PostgreSQL adapter for Python.

MySQL: Consider [mysql-connector-python](#) (the official Oracle driver) or [PyMySQL](#) (a pure-Python client).

These libraries allow you to execute the DDL and DML scripts required to load your dataset and perform complex analytical joins efficiently.

This project leverages a pre-processed version of the **Zomato dataset** to handle order creation logic. You can access the complete, unrefined dataset via [this link](#).

Q1: Database Design and Analytics

You have been hired as the lead Data Scientist for a growing food delivery platform. The data engineering team has provided you with a large, flat CSV file containing restaurant data. Before you can build predictive models, you must design a robust relational database, load the data, and perform exploratory data analysis using SQL.

The raw data has these columns:

Column Name	Explanation
address	The physical street address of the restaurant.
name	The official name of the restaurant.



Column Name	Explanation
online_order	A binary flag indicating if the restaurant accepts online orders (e.g., Yes/No).
book_table	A binary flag indicating if table booking is available at the restaurant (e.g., Yes/No).
rate	The aggregated customer rating score for the restaurant.
votes	The total number of customer ratings or votes received.
phone	The contact phone number for the restaurant.
location	The specific neighborhood or area where the restaurant is physically situated.
rest_type	The operational category or type of establishment (e.g., Quick Bite, Casual Dining).
cuisines	The types of food or cuisine offered by the restaurant (e.g., North Indian, Chinese).
approx_cost (for two people)	The estimated average cost for a meal for two people.
menu_item	A field listing the items available on the restaurant's menu.
listed_in (type)	The service type categorization for the listing (e.g., Buffet, Delivery).
listed_in (city)	The major city in which the restaurant is listed.

Database Design (30 Points)

Your first task is to define a proper database table to house this flat dataset. You must assign optimal data types and identify a primary key.

- **Entity Relationship Design:** Map the flat columns into at least three logical, distinct tables (e.g., Restaurants, Locations, Services/Metrics). List the columns that belong to each table.
- **Keys & Relationships:** Clearly identify the Primary Key (PK) for each table. Where tables connect, identify the Foreign Keys (FK) and the type of relationship (1-to-Many, Many-to-Many).
- **DDL Execution:** Write the SQL CREATE TABLE statements for your designed schema.

Data Loading (10 Points)

- **Data Ingestion:** Write a conceptual SQL script demonstrating how you would load the data from the raw CSV file.

SQL

```
INSERT INTO table_name (columns)
SELECT
    column_names...
FROM csv_file;
```

Business Analytics Queries (60 Points)

Write optimized SQL queries based on your newly designed relational schema to answer the following questions.

- Write a query to find the total number of restaurants and the average approximate cost for two people, filtering only for restaurants that offer "Delivery" (using `listed_in(type)`).

- Does allowing online orders affect pricing? Write a query to calculate the average approx_cost (for two people) and the total number of votes grouped by online_order status (Yes/No), broken down by listed_in(city).
- The marketing team wants to highlight top-tier dining neighborhoods. Write a query to find location neighborhoods that have an average restaurant rating (rate) strictly greater than 4.2 AND have at least 50 restaurants in that specific location.
- We want to build a pricing tier dashboard. Write a query that categorizes restaurants into three tiers based on approx_cost (for two people): 'Budget' (<=500), 'Mid-Range' (500-1000), 'Premium' (>=1000). Return city, tier, count, and average rating. Order by city (ASC) and count (DESC).

Q2: Kafka Consumer Implementation (100 pts)

Implement a Kafka consumer that connects to the Kafka server and reads order events from the **ashpaz.order** topic. The consumer must perform fast, stateless validation on each incoming order event. No database access is allowed at this stage.

Based on the validation result, the consumer must route each event to the appropriate Kafka topic:

- Valid orders → ashpaz.valid
- Invalid orders → ashpaz.error_log

For each invalid order, include the order ID, an error type, a detailed error reason, and the original order payload in the message written to **ashpaz.error_log**.



Order Event Source and Format

A Kafka producer is provided to you, which is responsible for generating and publishing order events to the **ashpaz.order** topic. All incoming order messages follow the format below:

JSON

```
{
  "order_id": "string",
  "user_id": "string",
  "restaurant_id": "string",
  "restaurant_name": "string",
  "restaurant_city": "string",
  "cuisines": [
    "string", ...
  ],
  "phone_number": "string",
  "request_online": true,
  "request_table": false,
  "order_time": "ISO-8601 UTC timestamp",
  "items": [
    {
      "item_id": "string",
      "unit_price": number,
      "quantity": integer
    }, ...
  ],
  "order_price": number
}
```

Note that `items` contains multiple ordered items and must be used to compute the total price.

Validation Rules

Rule	Validation Logic	Error Type
Phone Number Format	Phone number must start with +91 or 080	INVALID_PHONE
Order Mode Conflict	NOT (request_online AND request_table)	ORDER_MODE_CONFLICT
Order Price Correctness	$order_price == \sum(unit_price \times quantity)$	PRICE_MISMATCH

Error Event Format

Messages written to the **ashpaz.error_log** topic must follow this structure:

JSON

```
{
  "order_id": "string",
  "error_type": "string",
  "error_reason": ["list", "of", "error_types"],
  "original_order": { "full_order_payload" }
}
```

The `original_order` field must contain the complete, unmodified order message as consumed from the `ashpaz.order` topic. If multiple errors occur simultaneously, the `error_type` field should be set to `MULTI`.

Output Requirement

Display (log or print) at least one processed message to demonstrate that the consumer is running correctly and handling data.

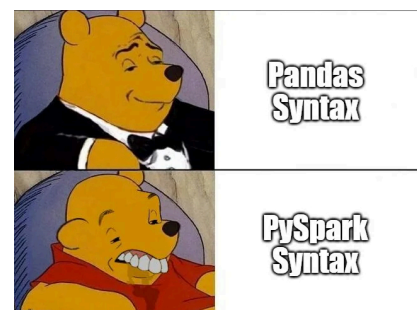
Questions:

- I. Apache Kafka is used as the ingestion layer for real-time food orders instead of writing orders directly into a relational database. Explain this design choice by comparing Kafka and relational databases in terms of performance, scalability, data access patterns, and suitability for real-time streaming, and briefly mention one scenario where a relational database would still be the better choice.
- II. In Q2, the Kafka consumer performs stateless validation and explicitly avoids database access. What is the difference between stateless and stateful processing? Why is stateless processing preferred at the ingestion layer, and what risks arise if Kafka consumers become state-dependent?
- III. Kafka is often described as a distributed commit log rather than a traditional message queue. Explain what this means in practice and how this design enables message replay, multiple independent consumers, and fault tolerance.

Q3: Pyspark Processing Layer(100 pts)

Batch Processing Layer (50 pts)

In this part of the project, the focus is on processing and analyzing existing data that has already been generated. This data is provided for you in the **historical_data.json** file. While the data processing and analytics must be implemented exclusively using **PySpark**, you are



welcome to use standard Python plotting libraries (such as Matplotlib, Seaborn, or Plotly,...) to visualize your final insights.

Revenue and Menu Analysis Batch Job(15 pts)

- I. **Top Ordered Items:** Process historical order data by extracting and counting the individual elements within the `items` arrays to identify and rank the most frequently ordered food items across all restaurants.
- II. **Highest Revenue Restaurants:** Aggregate the total `order_price` grouped by `restaurant_id` and `restaurant_name` to calculate total earnings, identifying the top-performing restaurants on the platform.
- III. **Order Channel Leaders (Online vs. Dine-In):** Aggregate transaction volumes grouped by `restaurant_id` to simultaneously count digital/delivery orders (`request_online` is true) and in-person dining traffic (`request_table` is true). This comparative analysis highlights which restaurants dominate the online delivery space and which are the top destinations for physical dine-in experiences.

Order Time & Peak Hour Analysis(20 pts)

- I. **Hourly Order Patterns:** Process historical order data by extracting the specific hour from the `order_time` field for each transaction. Group the transactions by these hourly intervals to calculate the total volume of orders placed throughout a standard 24-hour cycle.
- II. **Peak and Off-Peak Identification:** Analyze the aggregated hourly volumes to identify clear temporal patterns. Specifically, pinpoint the "most crowded" hours (highest order volume, representing daily rushes) and the "least crowded" hours (lowest order volume, representing quiet periods) to understand exactly when customer demand spikes and drops.

Geographic & Location-Based Analysis(15 pts)

- I. **Revenue Distribution by City:** Group historical orders by `restaurant_city` and aggregate the total `order_price` to identify which cities or regions generate the highest overall revenue.
- II. **Regional Favorite Foods (Menu Preference):** Extract and count the individual items sold within each `restaurant_city`. Rank these items to determine the most popular or "favorite" food in every city, helping restaurants tailor their regional menus.

Real-Time Processing Layer (50 pts)

In this part of the project, the focus is on processing **incoming data** in real-time as it arrives, rather than working with pre-existing or historical data in the dataset. The goal is to analyze the stream of new transactions from Kafka continuously.

For the sake of demonstration, order generation in the Ashpaz producer runs at an accelerated pace. We configured the pipeline so that every real-world second equals five minutes of simulated time. This design choice makes it much easier to observe temporal changes and test time-based logic in the streaming data without having to run the system for hours. If needed, you can adjust this pace at any time using the `TIME_ACCELERATION` parameter in the producer code.

Spark Streaming Application

- I. Implement a Spark Streaming application that connects to Kafka Consumer.
- II. Process data in small time intervals (micro-batches), using time-based windows to aggregate and analyze data over specific periods (e.g., 1 minute), with the window sliding at regular intervals (e.g., every 20 seconds), to ensure timely and efficient data processing in real time. Use the processing to gain some informative insights. Write detected insights to an (or multiple) arbitrary topic (topics).
- III. Implement a **checkpoint mechanism** for fault tolerance.

Real-Time Fraud Detection System(25 pts)

- I. **Geographical Impossibility:** Implement a streaming rule to monitor the `restaurant_city` for incoming orders. Flag any instance where the same `user_id` places orders in two different cities within an impossibly short time window (30 minutes in simulation time interval), indicating potential account compromise.
- II. **Velocity Check (Order Spam):** Implement a time-based windowing function to track order frequency. Identify and flag any scenario where a single `user_id` successfully places more than 5 separate orders within a strict 60-minute simulation time interval, which could indicate bot activity or a system glitch.
- III. **Alert Generation:** Write all flagged transactions that violate these rules, along with the triggering reason, to a dedicated Kafka topic (e.g., `orders.fraud_alerts`) for immediate isolation and review.

Real-Time Operational Analytics(25 pts)

- I. **Top Revenue by Cuisine:** Aggregate the total `order_price` grouped strictly by cuisine type across all orders. This will show you which food category (e.g., Italian, Fast Food, Persian) generates the highest total revenue platform-wide, regardless of where the restaurants are located.
- II. **Average Order Value per Cuisine:** Calculate the average `order_price` for each cuisine category globally. This helps identify if certain food types consistently result in larger check sizes (for example, fine-dining seafood might have a much higher AOV than casual burger joints, even if the total revenue is lower).

Questions:

- I. Explain the concept of Lazy Evaluation in Spark. Why does Spark wait until an Action (like `count()` or `save()`) is called before executing Transformations (like `filter()` or `groupBy()`)?

- II. During the ingestion phase, you implemented a consumer to validate order price correctness and phone formats. Why is it a "best practice" to perform this validation at the **Ingestion Layer (Kafka)** rather than waiting until the data reaches the **Batch Processing Layer (Spark)**?
- III. In a high-volume platform like Ashpaz, why is it dangerous if an order is processed twice? Discuss how **Checkpointing** in Spark and **Offsets** in Kafka work together to try and achieve **Exactly-Once** processing semantics.

Notes

Upload your work as a zip file in this format on the website: DS_CA2_[Std number].zip.

If the project is done in a group, include all of the group members' student numbers in the name.

Only one member must upload the work if the project is done in a group.

We will run your code during the project delivery, so make sure your results are reproducible¹.

Written answers to the questions are **not required**. These questions are for your own preparation; we will discuss these concepts and your design choices during the project delivery.

¹ Your code will be executed as-is in a fresh environment (e.g., a clean Docker container or VM) to verify that it runs without errors, and your outputs (e.g. visualizations and dataframes) are same (or effectively the same) as the result you've uploaded. Use practices like randomizing with seeds, to make the result more reproducible, but we also understand that due to the design of the project, reproducing exact same output on every section is not possible.

**Managing a
Kafka, easy!**

**The cloud
provider**

Engineer

Let's go! In and out. 20 minute adventure.

5 months later ...

(adult swim)

Supermeme.ai